

Desenvolvimento e teste de serviço de sistema no android

CURSO

FORMAÇÃO EM SISTEMAS EMBARCADOS

LEANDRO MENDES DOS SANTOS

Links

Link do Repositório: [gitHub](#)

Link da Aplicação: [VehicleEqualizerApp](#)

Link do Vídeo: [Vídeo](#)

OBJETIVOS DE APRENDIZAGEM

- Implementar um serviço de sistema Android que utilize JNI e JUNIT;
 - Implementar testes unitários para verificação do funcionamento do serviço no sistema Android;
 - Integrar outros componentes ao serviço do sistema Android;
 - Elaborar testes apresentando os resultados obtidos.
-

Introdução

O foco deste relatório é apresentar a implementação dos testes das classes devolvidas no relatório [anterior](#). Nas seções a seguir descreverei todos os testes e suas respectivas configurações.

1. [Configurações e Dependências](#)
 - 1.1. [Estrutura de Pastas](#)
2. [Testes Implementados](#)
 - 2.1. [AudioServiceTest](#)
 - 2.2. [EqualizerModuleTest](#)
 - 2.3. [NotificationModuleTest](#)
 - 2.4. [PlaybackModuleTest](#)
 - 2.5. [NativeIntegrationTest](#)
 - 2.6. [Resultados](#)
 - 2.7. [EqualizerNativeTest](#)
3. [Conclusão](#)

1. Configurações e Dependências

1.1 Estrutura de Pastas

Estrutura de pasta dos testes implementados para validação das funcionalidades do aplicativo.

```
.
├── androidTest # Departamento de testes instrumentados
│   ├── cpp # Valida código nativo escrito em C++ com JNI
│   │   ├── CMakeLists.txt
│   │   ├── EqualizerNativeTest.cpp
│   │   └── include
│   │       └── FakeJNIEnv.h
│   └── java
│       └── com
│           └── leandromendes
│               └── vehicleequalizer # Valida módulos e serviço de áudio do
aplicativo.
│                   ├── AudioServiceTest.kt
│                   ├── EqualizerModuleTest.kt
│                   └── NativeIntegrationTest.kt # Valida integração com
código nativo.
│                       ├── NotificationModuleTest.kt
│                       └── PlaybackModuleTest.kt
├── main
└── test #Diretório de testes unitários locais
    ├── java
    │   └── com
    │       └── leandromendes
    │           └── vehicleequalizer # Valida lógica da classe repositório.
    │               ├── stubs
    │               │   └── FakeProfileDao.kt
    │               └── UserRepositoryTest.kt
```

Abaixo as mudanças realizadas no arquivo `build.gradle.kts(:app)`, em resumo, adição de dependências e pequenos ajustes de versão. O arquivo está todo comentado para facilitar o entendimento.

```
...

dependencies {
    ...

    // Local Unit Tests (src/test) - JUnit 5
    // -----
    testImplementation(platform(libs.junit.bom))
    testImplementation(libs.junit.jupiter.api)
    testImplementation(libs.junit.jupiter.params)
    testImplementation(libs.junit.jupiter.engine)

    // Architecture and Coroutines
    testImplementation(libs.androidx.core.testing)
    testImplementation(libs.kotlinx.coroutines.test)
```

```

// Mocking Tools
testImplementation(libs.mockito.core)

// Instrumented Testing (src/androidTest)
// -----
androidTestImplementation(libs.androidx.core)
androidTestImplementation(libs.core.ktx)
androidTestImplementation(libs.androidx.runner)
androidTestImplementation(libs.androidx.rules)

// JUnit e Espresso
androidTestImplementation(libs.androidx.junit)
androidTestImplementation(libs.androidx.espresso.core)

// Mockito for instrumented testing
androidTestImplementation(libs.mockito.kotlin)
androidTestImplementation(libs.mockito.android)
}

// Ensures that all unit tests are run using the JUnit 5 testing framework
tasks.withType<Test>().configureEach {
    useJUnitPlatform()
}

...

```

Essas foram as principais alterações para a execução dos testes. No decorrer deste relatório, vou apresentar outra seção do arquivo X onde configuro os testes do código nativo.

2. Testes Implementados

Para não ficar muito grande, vou colocar apenas as partes dos testes de cada arquivo. Qualquer dúvida, consultar o arquivo completo conforme o diretório de pastas informado acima.

2.1. AudioServiceTest

[aosp/src/apps/VehicleEqualizerFull/app/src/androidTest/java/com/leandromendes/vehicleequalizer/AudioServiceTest.kt](#)

```

...

@Before
fun setup() {
    context = ApplicationProvider.getApplicationContext()
    serviceIntent = Intent(context, AudioService::class.java)
}

```

```

// Life Cycle Testing
// -----

@Test
fun service_bindsAndCallsOnCreate() {
    // Start the service by calling onCreate() and onBind().
    val binder = serviceRule.bindService(serviceIntent)

    // Verifies that the service was created correctly (the binder is
not null)
    assertNotNull(binder)
}

@Test
fun service_startsForegroundOnCreate() {
    // When starting the service, it must call startForeground.

    // bindService() returns the IBinder (AudioService.LocalBinder)
    val binder = serviceRule.bindService(serviceIntent)

    // Cast the binder to custom LocalBinder class
    val localBinder = binder as AudioService.LocalBinder

    // Retrieve the actual AudioService instance from the binder
    val service = localBinder.getService()
    assertNotNull(service)

    // We check the return value to ensure that the command was
processed.
    val result = service.onStartCommand(null, 0, 0)
    assertEquals(Service.START_STICKY, result)
}

// --- Reproduction Action Tests (Playback Actions) ---
// -----

@Test
fun onStartCommand_actionPlay_shouldStartPlayback() {
    // bindService() returns the IBinder (AudioService.LocalBinder)
    val binder = serviceRule.bindService(serviceIntent)

    // Cast the binder to custom LocalBinder class
    val localBinder = binder as AudioService.LocalBinder

    // Retrieve the actual AudioService instance from the binder
    val service = localBinder.getService()

    // Creates the ACTION_PLAY intent
    val playIntent = Intent(serviceIntent).apply {
        action = MusicConstants.ACTION_PLAY
    }

    // Execute the command
    service.onStartCommand(playIntent, 0, 1)
}

```

```

        // Checks the return value of onStartCommand and the lifecycle
state,
        // which should be START_STICKY
        val result = service.onStartCommand(playIntent, 0, 1)
        assertEquals(Service.START_STICKY, result)
    }

    @Test
    fun onStartCommand_actionPause_shouldPausePlayback() {
        // Starts and sets to PLAYING state (to ensure that PAUSE does
something)

        // bindService() returns the IBinder (AudioService.LocalBinder)
        val binder = serviceRule.bindService(serviceIntent)

        // Cast the binder to custom LocalBinder class
        val localBinder = binder as AudioService.LocalBinder

        // Retrieve the actual AudioService instance from the binder
        val service = localBinder.getService()

        service.onStartCommand(Intent(serviceIntent).apply { action =
MusicConstants.ACTION_PLAY }, 0, 1)

        // Creates the ACTION_PAUSE intent
        val pauseIntent = Intent(serviceIntent).apply {
            action = MusicConstants.ACTION_PAUSE
        }

        // Execute the command
        service.onStartCommand(pauseIntent, 0, 2)

        // Check the status
        val result = service.onStartCommand(pauseIntent, 0, 2)
        assertEquals(Service.START_STICKY, result)
    }

    @Test
    fun onStartCommand_actionNext_shouldAdvanceTrackAndPlay() {
        // bindService() returns the IBinder (AudioService.LocalBinder)
        val binder = serviceRule.bindService(serviceIntent)

        // Cast the binder to custom LocalBinder class
        val localBinder = binder as AudioService.LocalBinder

        // Retrieve the actual AudioService instance from the binder
        val service = localBinder.getService()

        // Check initial state
        assertEquals("Toto Africa", service.getCurrentTrackTitle())

        // Send the command to the service instance

```

```

        val nextIntent = Intent(serviceIntent).apply {
            action = MusicConstants.ACTION_NEXT
        }

        // Call the command directly on the service instance
        service.onStartCommand(nextIntent, 0, 1)

        // Verify the new state
        assertEquals("Yohan Kim Friends", service.getCurrentTrackTitle())
    }
    ...

```

2.2. EqualizerModuleTest

aosp/src/apps/VehicleEqualizerFull/app/src/androidTest/java/com/leandromendes/vehicleequalizer/EqualizerModuleTest.kt

```

...

@Before
fun setup() {
    context = ApplicationProvider.getApplicationContext()

    // Initializes MediaPlayer and obtains the actual audioSessionId
    val playbackModule = PlaybackModule(context)
    playbackModule.setRawDataSource(R.raw.toto_africa)
    playbackModule.play()

    val sessionId = playbackModule.getAudioSessionId()

    // Initializes the equalizer with the active audio session
    equalizerModule = EqualizerModule(context, sessionId)
}

@Test
fun testSetBandLevel() {
    val band = 0
    val level = -15
    equalizerModule.setBandLevelSafe(band, level)
    assertEquals("Incorrect band level", level.toShort(),
equalizerModule.getBandLevel(band))
}

...

```

2.3. NotificationModuleTest

aosp/src/apps/VehicleEqualizerFull/app/src/androidTest/java/com/leandromendes/vehicleequalizer/NotificationModuleTest.kt

```
...

@Before
fun setup() {
    context = ApplicationProvider.getApplicationContext()
    notificationModule = NotificationModule(context)
}

// Test the creation of the notification channel
@Test
fun testCreateNotificationChannel() {
    val notification: Notification =
notificationModule.buildNotification(
    playbackState = "PLAYING",
    trackTitle = "Test Music"
)
    assertNotNull("The notification shall not be void.", notification)
}

// Tests the display of a notification
@Test
fun testCreateNotification() {
    notificationModule.showNotification("PAUSED", "Test Music")
    assertTrue("Notification successfully displayed", true)
}

...
```

2.4. PlaybackModuleTest

aosp/src/apps/VehicleEqualizerFull/app/src/androidTest/java/com/leandromendes/vehicleequalizer/PlaybackModuleTest.kt

```
...

@Before
fun setup() {
    // Gets the application context and initializes the playback module
    context = ApplicationProvider.getApplicationContext()
    playbackModule = PlaybackModule(context)
}

@After
```

```

fun tearDown() {
    playbackModule.stop()
}

// Test audio playback
@Test
fun testPlayAudio() {
    playbackModule.setRawDataSource(R.raw.toto_africa)
    playbackModule.play()
    Assert.assertTrue("The audio should be playing after play()",
playbackModule.isPlaying())
    playbackModule.stop()
}

// Test the audio pause
@Test
fun testPauseAudio() {
    playbackModule.setRawDataSource(R.raw.toto_africa)
    playbackModule.play()
    playbackModule.pause()
    Assert.assertFalse("The audio should be paused after pause().",
playbackModule.isPlaying())
}

// Tests playback pause
@Test
fun testStopAudio() {
    playbackModule.setRawDataSource(R.raw.toto_africa)
    playbackModule.play()
    playbackModule.stop()
    Assert.assertFalse("The audio should be paused after stop()",
playbackModule.isPlaying())
}
...

```

2.5. NativeIntegrationTest

```

`aosp/src/apps/VehicleEqualizerFull/app/src/androidTest/java/com/leandromen
des/vehicleequalizer/NativeIntegrationTest.kt`

...

@Before
fun setup() {
    // Initializes the module before each test.
    equalizerModule =
EqualizerModule(ApplicationProvider.getApplicationContext(), 100)
    // Calls native initialization with a sample audio session ID.
    equalizerModule.setEqualizerEnabledNative(false)
}

```



```

        equalizerModule.setVolumeFromNative(50)
    }

    @Test
    fun testInitializationAndStatus() {
        val status = equalizerModule.getNativeStatus()
        // Checks whether initialization has occurred (default volume 50
and disabled).
        assertEquals("enabled=false, volume=50", status)
    }

    @Test
    fun testSetVolume() {
        val expectedVolume = 75
        equalizerModule.setVolumeFromNative(expectedVolume)
        // Verify that the volume has been set to the specified value.
        val status = equalizerModule.getNativeStatus()
        assertEquals("enabled=false, volume=$expectedVolume", status)
    }

    @Test
    fun testSetBandLevel() {
        val bandId = 1 // 910Hz
        val level = 12 // +12dB

        equalizerModule.setBandLevelNative(bandId, level)
        val retrievedLevel = equalizerModule.getBandLevelNative(bandId)
        // Check that the frequency band has been set to the correct value
        assertEquals(level, retrievedLevel)
    }

    @Test
    fun testEnableEqualizer() {
        equalizerModule.setEqualizerEnabledNative(true)
        val status = equalizerModule.getNativeStatus()
        // Check if the equalizer has been enabled
        assertTrue(status.contains("enabled=true"))
    }

    ...

```

2.6. Resultados

A imagem abaixo exibe o resultado da execução de todos os testes citados anteriormente.

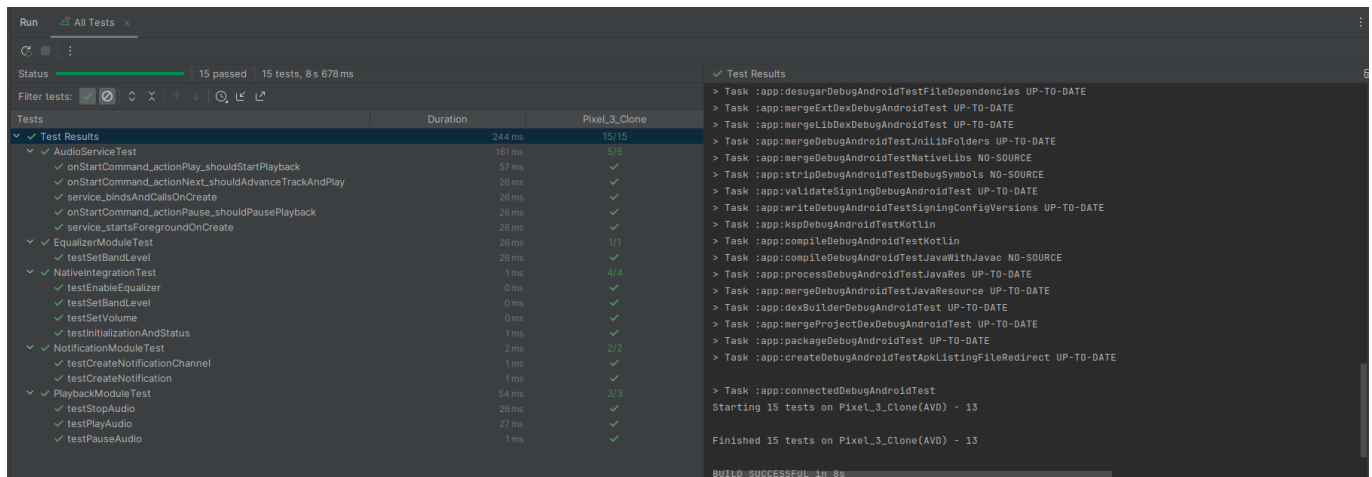


Figura 1: Resultado dos testes

2.7. EqualizerNativeTest

Agora, irei falar dos testes feitos para o código nativo em c++, também escritos em C++. A seguir descreverei todas as alterações e o resultado da execução.

- Alterações no CMakeList da native-lib.cpp

Linhas adicionadas no final do arquivo, apontando para o arquivo de teste.

`aosp/src/apps/VehicleEqualizerFull/app/src/main/cpp/CMakeLists.txt`

```
enable_testing()

# -----
# Adds native integration tests (AndroidTest)
set(TEST_DIR "${CMAKE_CURRENT_SOURCE_DIR}/../../androidTest/cpp")
if(EXISTS "${TEST_DIR}/EqualizerNativeTest.cpp")
    message(STATUS "Including native integration tests from ${TEST_DIR}")
    add_subdirectory(${TEST_DIR} ${CMAKE_CURRENT_BINARY_DIR}/androidTest)
endif()
```

- CmakeLists do arquivo contendo os testes:

`aosp/src/apps/VehicleEqualizerFull/app/src/androidTest/cpp/CMakeLists.txt`

```
cmake_minimum_required(VERSION 3.22.1)
project(vehicleequalizer_native_lib_tests LANGUAGES CXX)

# -----
# Includes GTest (using FetchContent)
# -----
include(FetchContent)
FetchContent_Declare(
    googletest
    URL https://github.com/google/googletest/archive/release-1.12.1.zip
```

```

)
FetchContent_MakeAvailable(googletest)

# -----
# Test source files
# -----
set(TEST_SRC
    EqualizerNativeTest.cpp
)

# -----
# Add test executable
# -----
add_executable(vehicleequalizer_native_lib_tests ${TEST_SRC})

# -----
# Include directories
# -----
target_include_directories(vehicleequalizer_native_lib_tests PRIVATE
    ${CMAKE_CURRENT_SOURCE_DIR}    # Para encontrar FakeJNIEnv.h
    ${gtest_SOURCE_DIR}/include
)

# -----
# Required libraries
# -----
target_link_libraries(vehicleequalizer_native_lib_tests
    gtest
    gtest_main
    log
    android
)

# -----
# Define C++17
# -----
set_target_properties(vehicleequalizer_native_lib_tests PROPERTIES
    CXX_STANDARD 17
    CXX_STANDARD_REQUIRED YES
)

```

- Alterações no arquivo `build.gradle.kts(:app)` para poder buildar e carregar os testes com as dependências dentro do emulador.

```

// -----
// Custom task for native testing
// -----
abstract class RunNativeTestsTask @Inject constructor(
    private val execOps: ExecOperations
) : DefaultTask() {

```

```

@get:InputDirectory
abstract val nativeBuildDir: DirectoryProperty

@get:Input
abstract val deviceTestDir: Property<String>

@get:Input
abstract val testBinaryName: Property<String>

@get:OutputDirectory
abstract val reportsDir: DirectoryProperty

// Log Section
private fun logSection(title: String) {
    val line = "=".repeat(60)
    println("\u001B[34m$line\n$title\n$line\u001B[0m")
}

private fun logStep(msg: String) = println("\u001B[32m➡️ $msg\u001B[0m")
private fun logInfo(msg: String) = println("\u001B[36m📄 $msg\u001B[0m")
private fun logError(msg: String) = println("\u001B[31m❌ $msg\u001B[0m")

@TaskAction
fun run() {
    logSection("RUNNING NATIVE TESTS 🚀")

    val buildDir = nativeBuildDir.get().asFile
    val abiList = listOf("x86_64", "arm64-v8a", "armeabi-v7a")

    // Detects device ABI
    val abiOutput = ByteArrayOutputStream()
    execOps.exec {
        commandLine("adb", "shell", "getprop", "ro.product.cpu.abi")
        standardOutput = abiOutput
    }
    val deviceAbi = abiOutput.toString().trim()

    // Select ABI available in the build
    val abi = if (buildDir.resolve(deviceAbi).exists()) deviceAbi
    else abiList.firstOrNull { buildDir.resolve(it).exists() }
    ?: error("No compatible binary found in the build directory")

    val abiDir = buildDir.resolve(abi)
    val binaryPath = abiDir.resolve(testBinaryName.get())
    val sharedLibPath = abiDir.resolve("libc++_shared.so")
    val deviceDir = deviceTestDir.get()
    val reportsPath = reportsDir.get().asFile.also { it.mkdirs() }
    val reportFileName = "report.xml"

    logInfo("ABI detected: $abi")
}

```

```

        logInfo("Biblioteca compartilhada: $binaryPath")
        logInfo("Shared library: $sharedLibPath")
        logInfo("Test directory on the device: $deviceDir")

        fun runCmd(vararg args: String) = logStep(args.joinToString("
"))).also {
            execOps.exec { commandLine(*args) }
        }

        try {
            // Runs the binary on the device
            runCmd("adb", "shell", "mkdir", "-p", deviceDir)
            runCmd("adb", "push", binaryPath.absolutePath, deviceDir)
            runCmd("adb", "push", sharedLibPath.absolutePath, deviceDir)
            runCmd("adb", "shell", "chmod", "777",
"$deviceDir/${testBinaryName.get()}")

            // Runs the binary on the device
            val output = ByteArrayOutputStream()
            execOps.exec {
                commandLine(
                    "adb", "shell",
                    "LD_LIBRARY_PATH=$deviceDir",
                    "$deviceDir/${testBinaryName.get()}",
                    "--gtest_output=xml:$deviceDir/$reportFileName"
                )
                standardOutput = output
                errorOutput = output
                isIgnoreExitValue = true
            }

            // Pulls report back to host
            runCmd("adb", "pull", "$deviceDir/$reportFileName",
reportsPath.absolutePath)

            logSection("NATIVE TEST OUTPUT 📄")
            println(output.toString(Charsets.UTF_8))
            logSection("REPORT SAVED ✅")
            logInfo("Report saved in:
${reportsPath.resolve(reportFileName).absolutePath}")

        } catch (e: Exception) {
            logError("Error running native tests: ${e.message}")
            throw e
        }
    }
}

// Task registration
tasks.register<RunNativeTestsTask>("runNativeTests") {
    group = "verification"
    description = ""Runs native C++ tests on the Android device""

```

```

nativeBuildDir.set(layout.buildDirectory.dir("intermediates/cmake/debug/obj
"))
    deviceTestDir.set("/data/local/tmp/gtests/${project.name}")
    testBinaryName.set("vehicleequalizer_native_lib_tests")
    reportsDir.set(layout.buildDirectory.dir("test-results/native"))

    dependsOn("externalNativeBuildDebug")
    outputs.upToDateWhen { false }
}

```

- Arquivos contendo os testes:

aosp/src/apps/VehicleEqualizerFull/app/src/androidTest/cpp/EqualizerNativeTest.cpp

```

...

#define TAG "NativeIntegrationTest"
#define LOGI(...) __android_log_print(ANDROID_LOG_INFO, TAG, __VA_ARGS__)

// -----
// Testes GTest
// -----
class EqualizerNativeTest : public ::testing::Test {
protected:
    FakeJNIEnv env;
    jobject dummy = nullptr;

    void SetUp() override {

        Java_com_leandromendes_vehicleequalizer_modules_equalizer_EqualizerModule_i
        nitialization(&env, dummy, 0);
    }

    std::string getStatus() {

        Java_com_leandromendes_vehicleequalizer_modules_equalizer_EqualizerModule_g
        etNativeStatus(&env, dummy);
        return env.captured_status;
    }
};

TEST_F(EqualizerNativeTest, InitialStateIsCorrectAfterSetup) {
    ASSERT_EQ(getStatus(), "enabled=false, volume=50");
    LOGI("Initial State test passed.");
}

TEST_F(EqualizerNativeTest, SetVolumeChangesStateCorrectly) {

    Java_com_leandromendes_vehicleequalizer_modules_equalizer_EqualizerModule_s

```

```

etVolumeFromNative(&env, dummy, 75);
    ASSERT_EQ(getStatus(), "enabled=false, volume=75");
    LOGI("Set Volume test passed.");
}

TEST_F(EqualizerNativeTest, EnableEqualizerChangesStateCorrectly) {

Java_com_leandromendes_vehicleequalizer_modules_equalizer_EqualizerModule_s
etEqualizerEnabledNative(&env, dummy, JNI_TRUE);
    ASSERT_EQ(getStatus(), "enabled=true, volume=50");
    LOGI("Enable Equalizer test passed.");
}

TEST_F(EqualizerNativeTest, CombinedStateChangeIsCorrect) {

Java_com_leandromendes_vehicleequalizer_modules_equalizer_EqualizerModule_s
etVolumeFromNative(&env, dummy, 100);

Java_com_leandromendes_vehicleequalizer_modules_equalizer_EqualizerModule_s
etEqualizerEnabledNative(&env, dummy, JNI_TRUE);
    ASSERT_EQ(getStatus(), "enabled=true, volume=100");
    LOGI("Combined State test passed.");
}

```

- Execução e resultados:

↳ `./gradlew runNativeTests` # Comando para iniciar os testes

> Task :app:runNativeTests

=====

RUNNING NATIVE TESTS 🚀

=====

📘 ABI detected: x86_64

📘 Biblioteca compartilhada:

VehicleEqualizerFull/app/build/intermediates/cmake/debug/obj/x86_64/vehicle
equalizer_native_lib_tests

📘 Shared library:

VehicleEqualizerFull/app/build/intermediates/cmake/debug/obj/x86_64/libc++_
shared.so

📘 Test directory on the device: /data/local/tmp/gtests/app

➡ adb shell mkdir -p /data/local/tmp/gtests/app

➡ adb push

VehicleEqualizerFull/app/build/intermediates/cmake/debug/obj/x86_64/vehicle
equalizer_native_lib_tests /data/local/tmp/gtests/app

VehicleEqualizerFull/app/build/intermediates/cmake/debug/obj/x86_64/vehicle
equalizer_native_lib_tests: 1 file pushed. 272.8 MB/s (3730656 bytes in
0.013s)

➡ adb push

VehicleEqualizerFull/app/build/intermediates/cmake/debug/obj/x86_64/libc++_
shared.so /data/local/tmp/gtests/app

```

VehicleEqualizerFull/app/build/intermediates/cmake/debug/obj/x86_64/libc++_shared.so: 1 file pushed. 259.2 MB/s (1617608 bytes in 0.006s)
➡ adb shell chmod 777
/data/local/tmp/gtests/app/vehicleequalizer_native_lib_tests
➡ adb pull /data/local/tmp/gtests/app/report.xml
VehicleEqualizerFull/app/build/test-results/native
/data/local/tmp/gtests/app/report.xml: 1 file pulled. 2.9 MB/s (1707 bytes in 0.001s)

=====
NATIVE TEST OUTPUT
=====

Running main() from
VehicleEqualizerFull/app/.cxx/Debug/395h4p3t/x86_64/_deps/googletest-src/googletest/src/gtest_main.cc
[=====] Running 4 tests from 1 test suite.
[-----] Global test environment set-up.
[-----] 4 tests from EqualizerNativeTest
[ RUN      ] EqualizerNativeTest.InitialStateIsCorrectAfterSetup
[          OK ] EqualizerNativeTest.InitialStateIsCorrectAfterSetup (0 ms)
[ RUN      ] EqualizerNativeTest.SetVolumeChangesStateCorrectly
[          OK ] EqualizerNativeTest.SetVolumeChangesStateCorrectly (0 ms)
[ RUN      ] EqualizerNativeTest.EnableEqualizerChangesStateCorrectly
[          OK ] EqualizerNativeTest.EnableEqualizerChangesStateCorrectly (0 ms)
[ RUN      ] EqualizerNativeTest.CombinedStateChangeIsCorrect
[          OK ] EqualizerNativeTest.CombinedStateChangeIsCorrect (0 ms)
[-----] 4 tests from EqualizerNativeTest (0 ms total)

[-----] Global test environment tear-down
[=====] 4 tests from 1 test suite ran. (0 ms total)
[ PASSED   ] 4 tests.

=====
REPORT SAVED ✓
=====

i Report saved in: aosp/src/apps/VehicleEqualizerFull/app/build/test-results/native/report.xml

BUILD SUCCESSFUL in 2s
10 actionable tasks: 10 executed

```

- Abrindo o reporte no Android Studio

Basta ir para tela de Run no Android Studio e importar o arquivo Report.xml gerado no final do teste

Report saved in: aosp/src/apps/VehicleEqualizerFull/app/build/test-results/native/report.xml

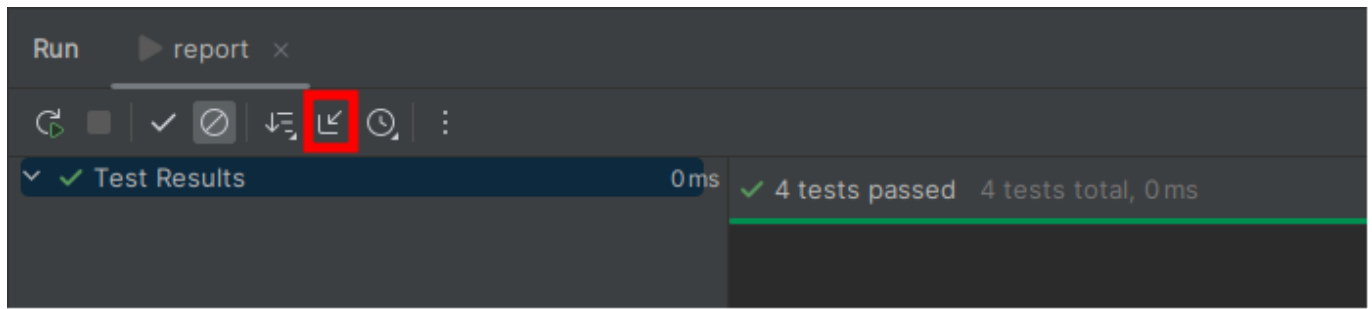


Figura 2: Resultado dos testes Nativos

3. Conclusão

Com isso, concluo toda a parte de testes. Os principais pontos de melhoria que posso observar seriam o mock de mais classes para aumentar a cobertura de testes, principalmente nas classes módulos, que são as que mais têm dependências externas. Sobre os testes nativos, integrar diretamente na interface no Android Studio para não ter de rodar pelo terminal.